

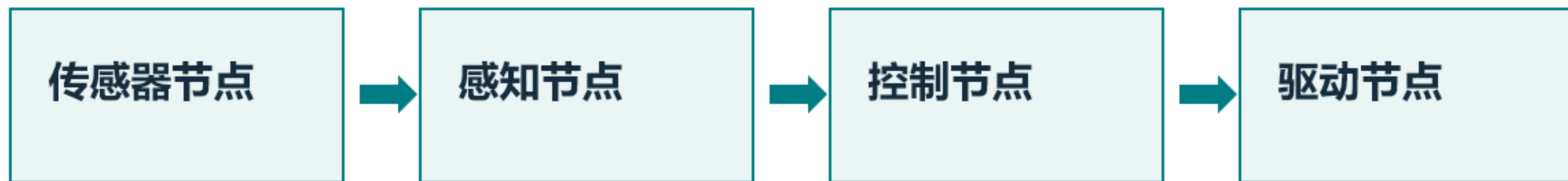
ROS 2

与小海龟控制

用 Python 节点发送运动指令

郑子毅 · 机协内训 Demo · 10 分钟

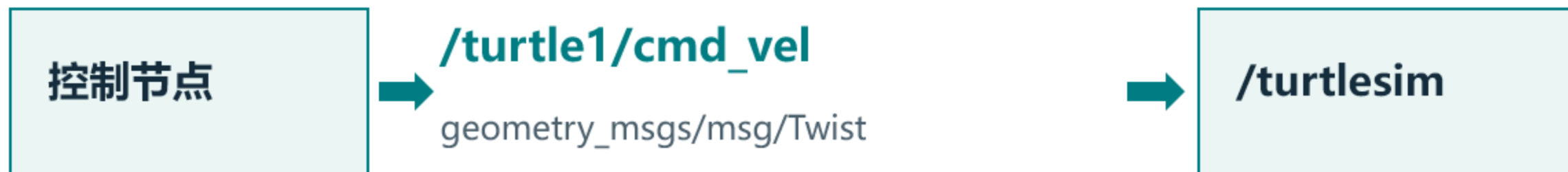
机器人软件中的消息协作



各节点负责清楚的任务，通过消息交换数据。

独立排查一个环节，能减少整套系统一起修改的成本。

节点、话题与消息



节点：执行功能的参与者

话题：发布者与订阅者约定的通信名称

消息：双方一致的数据类型和字段

话题、服务与动作

机制	适合的交互	机器人例子
话题 Topic	持续发布数据	速度指令、位姿
服务 Service	请求与响应	重置仿真、查询状态
动作 Action	长期目标与反馈	导航目标与取消

本 Demo 使用话题。一次请求也不自动具有实时性保证。

第一条速度指令

```
source /opt/ros/jazzy/setup.bash
ros2 run turtlesim turtlesim_node

# 新终端加载相同环境, 再发布
ros2 topic pub --rate 10 /turtle1/cmd_vel \
  geometry_msgs/msg/Twist \
  "{linear: {x: 1.0}, angular: {z: 1.0}}"
```

linear.x 控制前进, angular.z 控制平面转向。
10 Hz 是发布频率, 速度大小由消息字段决定。

Python 发布节点的三个位置

```
class CircleNode(Node):  
    def __init__(self):  
        super().__init__('circle_controller')  
        self.pub = self.create_publisher(  
            Twist, '/turtle1/cmd_vel', 10)  
        self.timer = self.create_timer(0.1, self.tick)  
  
    def tick(self):  
        msg = Twist()  
        msg.linear.x = 1.0  
        msg.angular.z = 1.0  
        self.pub.publish(msg)
```

创建发布者、设置定时器、发布消息。
这是核心片段。完整示例包含导入、spin 与限时停止。

小海龟走圆的原因

保持线速度 v 与非零角速度 ω

圆的半径 $r = |v / \omega|$

$v = 1.0$, $\omega = 1.0$, 半径约为 1 个仿真坐标单位

相同 v 下, $|\omega|$ 越大, 转弯越紧

机器人不动时的排查

1 环境与节点

source 环境后, `ros2 node list`

2 话题与类型

`ros2 topic list -t`; 核对 Twist

3 消息与连接

`ros2 topic echo`; `ros2 topic info`

4 指令来源

关闭多余控制器, 核对域编号与 QoS

课内小挑战

保持 $v = 1.0$ ，让半径约为 2。

先预测 ω ，再修改参数观察轨迹。

演示结束时主动发布全零 Twist。

参考： $\omega = 0.5$ 或 -0.5 ，转向相反

验收与竞赛迁移

能解释节点、话题与消息的关系。

能改变圆的半径，观察消息，并主动停车。

能按 README 让另一位同学复现。

下一课加入位姿反馈，再学习实车驱动与安全停机。